NODEJS GPT Manual Code Review

Vulnerability 1

Vulnerability 1

CWE-284: Improper Access Control
Location: auth-service/src/controllers/auth.controller.ts

Evidence:

```
role: {
  connect: { name: validatedData.role }
}
```

in validators proof:

role: z.enum(['patron', 'staff', 'librarian'], { required_error: 'Role is required' })

(User can submit any of the 3)

The public registration endpoint allows the client to supply validatedData.role during account creation. Because there is no authorization check restricting role assignment, a user may be able to self-register as a elevated privileged role such as staff or librarian, interestingly enough, I saw this exact same issue in Flask GPT's version of the library app.

If a savvy attacker realizes this, it could be very devastating.

Threat: High

Vulnerability 2

CWE-20: Improper Input Validation
Location: book-catalog-service/src/services/book.service.ts

Evidence:

```
export const createBook = async (data: Partial<Book>): Promise<Book> => {
  return prisma.book.create({ data });
};
```

```
export const updateBook = async (id: string, data: Partial<Book>): Promise<Book> => {
  ...
  return prisma.book.update({ where: { id }, data });
};
```

The service accepts raw Partial<Book> input and passes it directly into persistence without schema validation or field allowlisting. Because the controller forwards req.body directly, client-controlled fields are not explicitly validated or constrained before database write operations.

Vulnerability 3

CWE-284: Improper Access Control
Location: book-request-service/src/controllers/request.controller.ts and book-request-service/src/services/request.service.ts

Evidence:

```
const { title, author, description, requestedBy } = req.body;
const request = await requestService.createRequest({
  title,
  author,
  description,
  requestedBy,
});
```

```
const { status, reviewedBy } = req.body;
const updated = await requestService.updateRequestStatus(
  id,
  status,
  reviewedBy
);
```

```
return await prisma.bookRequest.create({
  data,
});
```

```
return await prisma.bookRequest.update({
  where: { id },
  data: { status, reviewedBy },
});
```

The request workflow trusts client-controlled identity fields such as requestedBy and reviewedBy instead of getting them from the authenticated user context. Users can potentially act with elevated roles or on behalf of others and perform unauthorized actions.

Vulnerability X

CWE-200: Exposure of Sensitive Information
Generic returning of unsensitized errors directly to the client present in all of these systems.